

Loan Approval Classification Analysis

```
In [11]: import pandas as pd
import numpy as np
import xgboost as xgb
import matplotlib.pyplot as plt
import seaborn as sns
import optuna
import hvplot.pandas

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, train_test_split, KFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, roc_auc_score
from sklearn.pipeline import make_pipeline, Pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from xgboost import XGBClassifier
from sklearn.preprocessing import StandardScaler

from scipy.stats import uniform, loguniform, randint
```

Data Pre-Processing

```
In [12]: data = pd.read_csv("loan_data1.csv", engine="python")
```

```
In [13]: data = data.drop(columns=["loan_percent_income"]) # drop as redundant with person_income and loan_amnt
```

```
In [14]: missing = data.isnull().sum() #check for missing values
missing
```

```
Out[14]: person_age          0
person_gender          0
person_education       0
person_income          0
person_emp_exp         0
person_home_ownership  0
loan_amnt              0
loan_intent            0
loan_int_rate          0
cb_person_cred_hist_length  0
credit_score           0
previous_loan_defaults_on_file  0
loan_status            0
dtype: int64
```

```
In [15]: # Check for duplicate rows
data.duplicated().any()
```

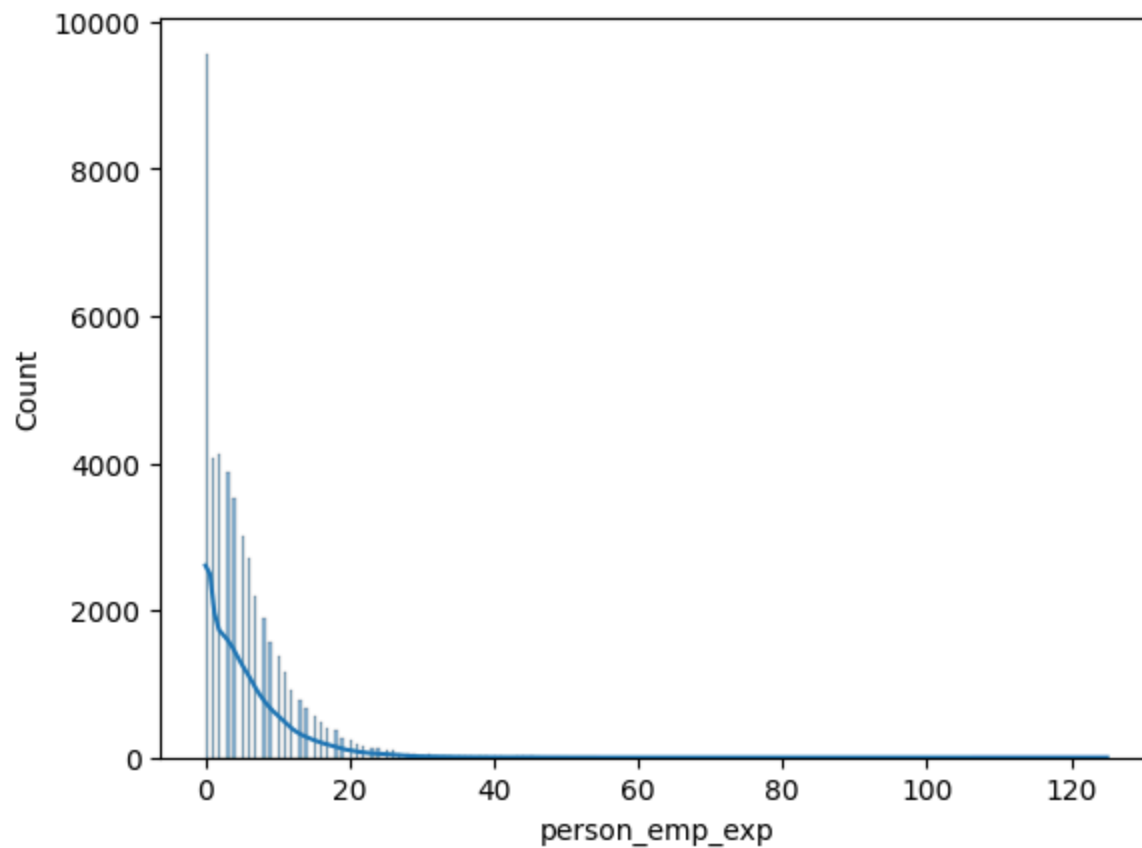
```
Out[15]: False
```

```
In [16]: # Check for obscure values in employment experience
data.person_emp_exp.describe()
```

```
Out[16]: count    45000.000000
          mean      5.410333
          std       6.063532
          min       0.000000
          25%       1.000000
          50%       4.000000
          75%       8.000000
          max      125.000000
          Name: person_emp_exp, dtype: float64
```

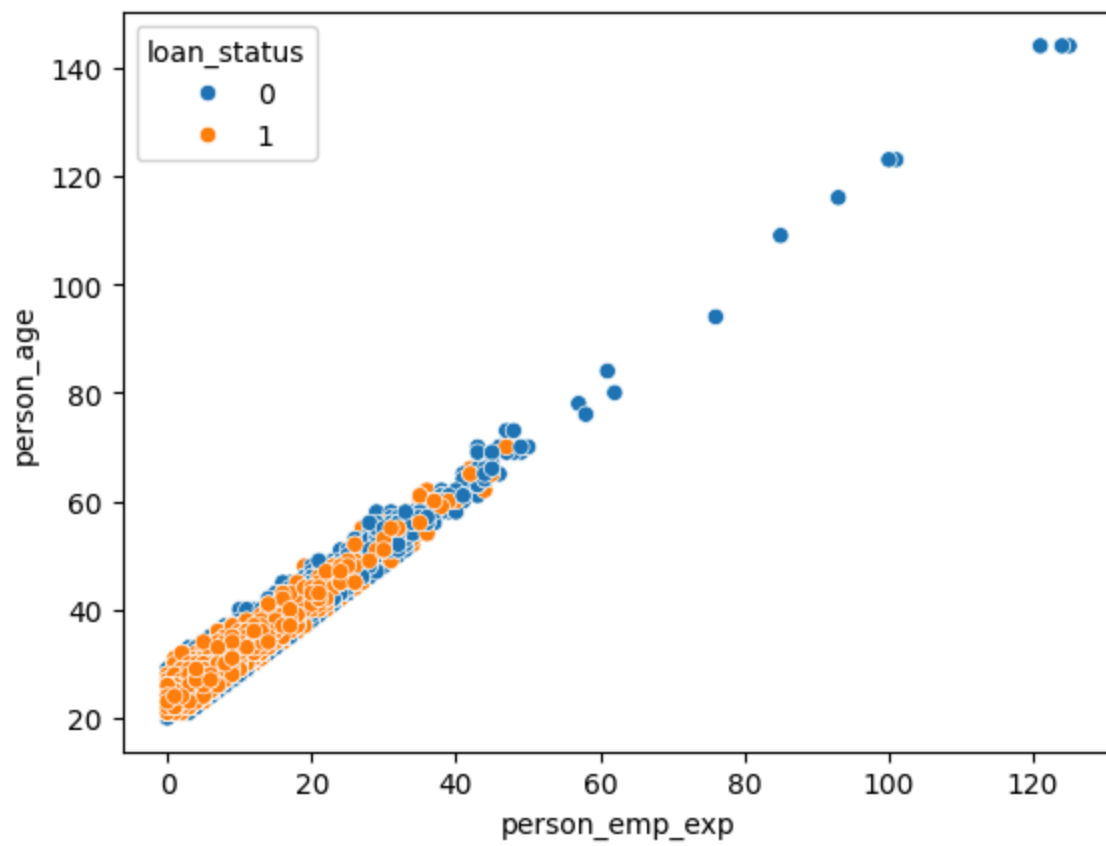
```
In [17]: sns.histplot(data.person_emp_exp, bins=300, kde=True)
```

```
Out[17]: <Axes: xlabel='person_emp_exp', ylabel='Count'>
```



```
In [18]: #Check the correlation between employment experience and age
          #We also can see the obscure values in employment experience
          sns.scatterplot(data=data, x='person_emp_exp', y = 'person_age', hue='loan_status')
```

```
Out[18]: <Axes: xlabel='person_emp_exp', ylabel='person_age'>
```



```
In [19]: plt.figure(figsize=(12,8))
sns.heatmap(data.corr(numeric_only=True), annot=True, fmt=".2f", cmap="viridis")
plt.show()
```



```
In [20]: # Look at rows with employment experience greater than 75 years
high_values_exp = data[data.person_emp_exp > 75]
high_values_exp
```

Out[20]:

	person_age	person_gender	person_education	person_income	person_emp_exp	person_home_owr
81	144.0	male	Bachelor	300616.0	125	
183	144.0	male	Associate	241424.0	121	MOF
575	123.0	female	High School	97140.0	101	
747	123.0	male	Bachelor	94723.0	100	
32297	144.0	female	Associate	7200766.0	124	MOF
32416	94.0	male	High School	29738.0	76	
37930	116.0	male	Bachelor	5545545.0	93	MOF
38113	109.0	male	High School	5556399.0	85	MOF

```
In [21]: # From above we can see there are extreme outliers in person_emp_exp and person_age which we will
# This are all very likely to be data entry errors and represent a very small number of rows so I
```

```
data = data[data.person_emp_exp < 75]
data = data[data.person_age < 75]
```

```
In [22]: data = data.loc[:, ~data.T.duplicated()] # drop duplicate columns if there are any

duplicate_rows = data.duplicated().sum()
print(f"Number of duplicate rows: {duplicate_rows}")
```

Number of duplicate rows: 0

```
In [23]: # Get a summary of the numerical columns to check for any other anomalies
data.describe()
```

```
Out[23]:
```

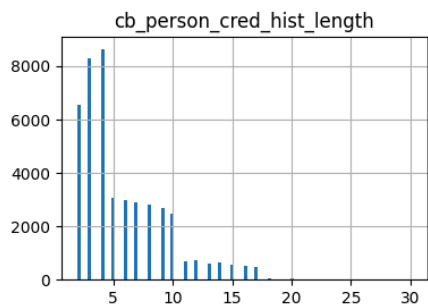
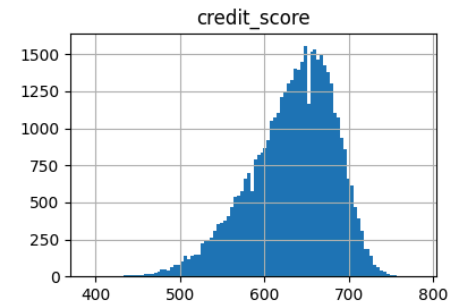
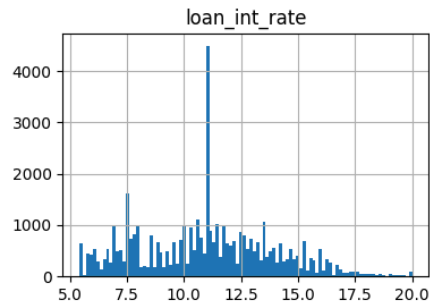
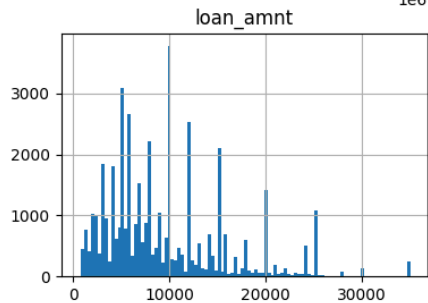
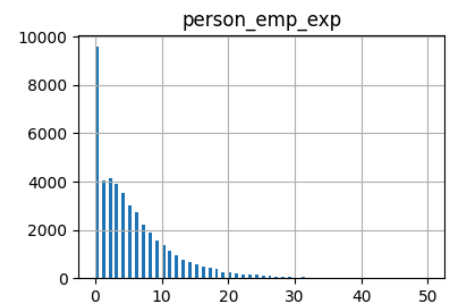
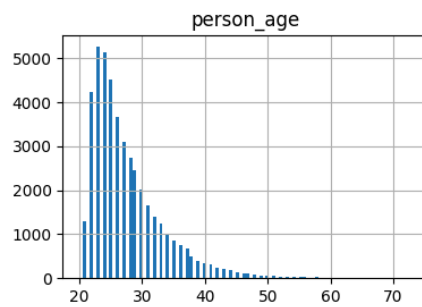
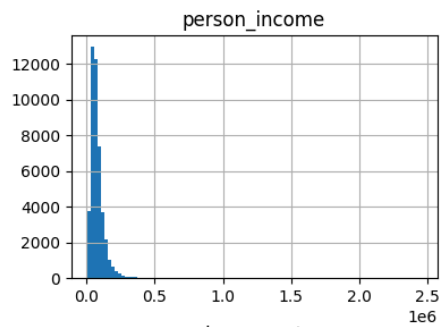
	person_age	person_income	person_emp_exp	loan_amnt	loan_int_rate	cb_person_cred_hist_ler
count	44988.000000	4.498800e+04	44988.000000	44988.000000	44988.000000	44988.000000
mean	27.742354	7.990907e+04	5.388148	9583.323820	11.006634	5.864
std	5.881527	6.332489e+04	5.896067	6314.994983	2.979042	3.872
min	20.000000	8.000000e+03	0.000000	500.000000	5.420000	2.000
25%	24.000000	4.719425e+04	1.000000	5000.000000	8.590000	3.000
50%	26.000000	6.704550e+04	4.000000	8000.000000	11.010000	4.000
75%	30.000000	9.577900e+04	8.000000	12237.250000	12.990000	8.000
max	73.000000	2.448661e+06	50.000000	35000.000000	20.000000	30.000



```
In [24]: # Visualize distributions of numerical variables
variables = ['person_income', 'person_age', 'person_emp_exp', 'loan_amnt', 'loan_int_rate', 'credit_score']

data[variables].hist(bins=100, figsize=(15,10))
```

```
Out[24]: array([[<Axes: title={'center': 'person_income'}>,
<Axes: title={'center': 'person_age'}>,
<Axes: title={'center': 'person_emp_exp'}>],
[<Axes: title={'center': 'loan_amnt'}>,
<Axes: title={'center': 'loan_int_rate'}>,
<Axes: title={'center': 'credit_score'}>],
[<Axes: title={'center': 'cb_person_cred_hist_length'}>, <Axes: >,
<Axes: >]], dtype=object)
```

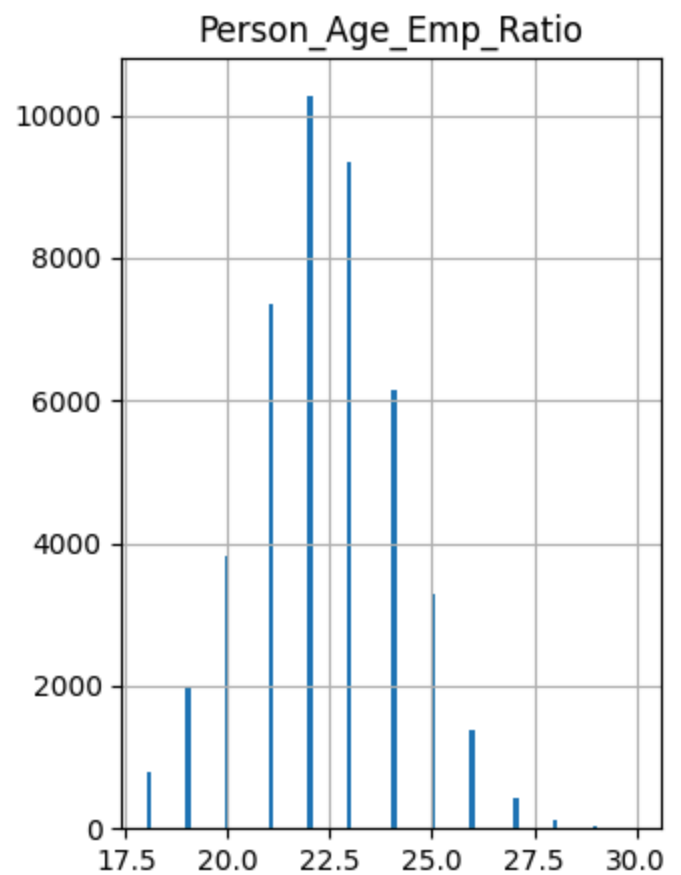
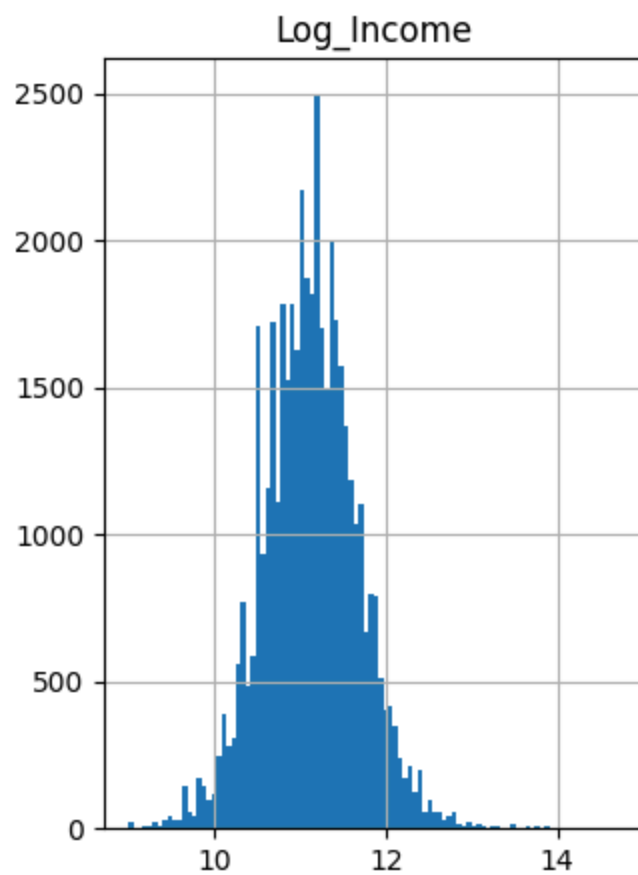


```
In [25]: data['Log_Income'] = np.log(data['person_income'] + 1) # Add 1 to avoid log(0)
data['Person_Age_Emp_Ratio'] = data['person_age'] - data['person_emp_exp']
```

```
In [26]: # Visualize distributions of transformed numerical variables
# We see the transformations have made the distributions more normal, and reduced a dimension
variables = ['Log_Income', 'Person_Age_Emp_Ratio']

data[variables].hist(bins=100, figsize=(8,5))
```

```
Out[26]: array([[<Axes: title={'center': 'Log_Income'}>,
<Axes: title={'center': 'Person_Age_Emp_Ratio'}>]], dtype=object)
```



In [27]: *# Check the distribution of the target variable*

```
data['loan_status'].value_counts()
```

Out[27]:

loan_status	
0	34988
1	10000
Name: count, dtype: int64	

Logistic Regression

Pre-Processing Data for Logistic Regression

In [47]: `data.describe(include=['object'])`

Out[47]:

	person_gender	person_education	person_home_ownership	loan_intent	previous_loan_defaults_on
count	44988	44988	44988	44988	44988
unique	2	5	4	6	2
top	male	Bachelor	RENT	EDUCATION	
freq	24832	13395	23436	9151	22956

In [48]: *# We will first map the categorical variables to dummy variables using the baseline value as the*

```
data1 = pd.get_dummies(data, drop_first=False, columns=['person_gender', 'person_education', 'perso
```

```
data1 = data1.drop(columns=['person_gender_male', 'person_education_High School', 'person_home_owne
data1 = data1.drop(columns=['person_emp_exp', 'person_income']) # drop due to high correlation a
```

```
In [30]: # Split our data, and specify our K-fold cross validation and logistic regression model

X, y = data1.drop(columns=["loan_status"]), data1["loan_status"]

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

log_reg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000, penalty='l2', solver='
```

```
In [49]: # Compute cross-validated accuracy scores for model

scores = cross_val_score(log_reg, X, y, cv=skf, scoring='accuracy')

print(f"Accuracies per fold: {scores}")
print(f"Mean CV Accuracy: {scores.mean():.4f} ± {scores.std():.4f}")
```

Accuracies per fold: [0.8950878 0.89864414 0.89308735 0.8945204 0.89674336]
Mean CV Accuracy: 0.8956 ± 0.0019

```
In [50]: # Get cross-validated predictions and confusion matrix

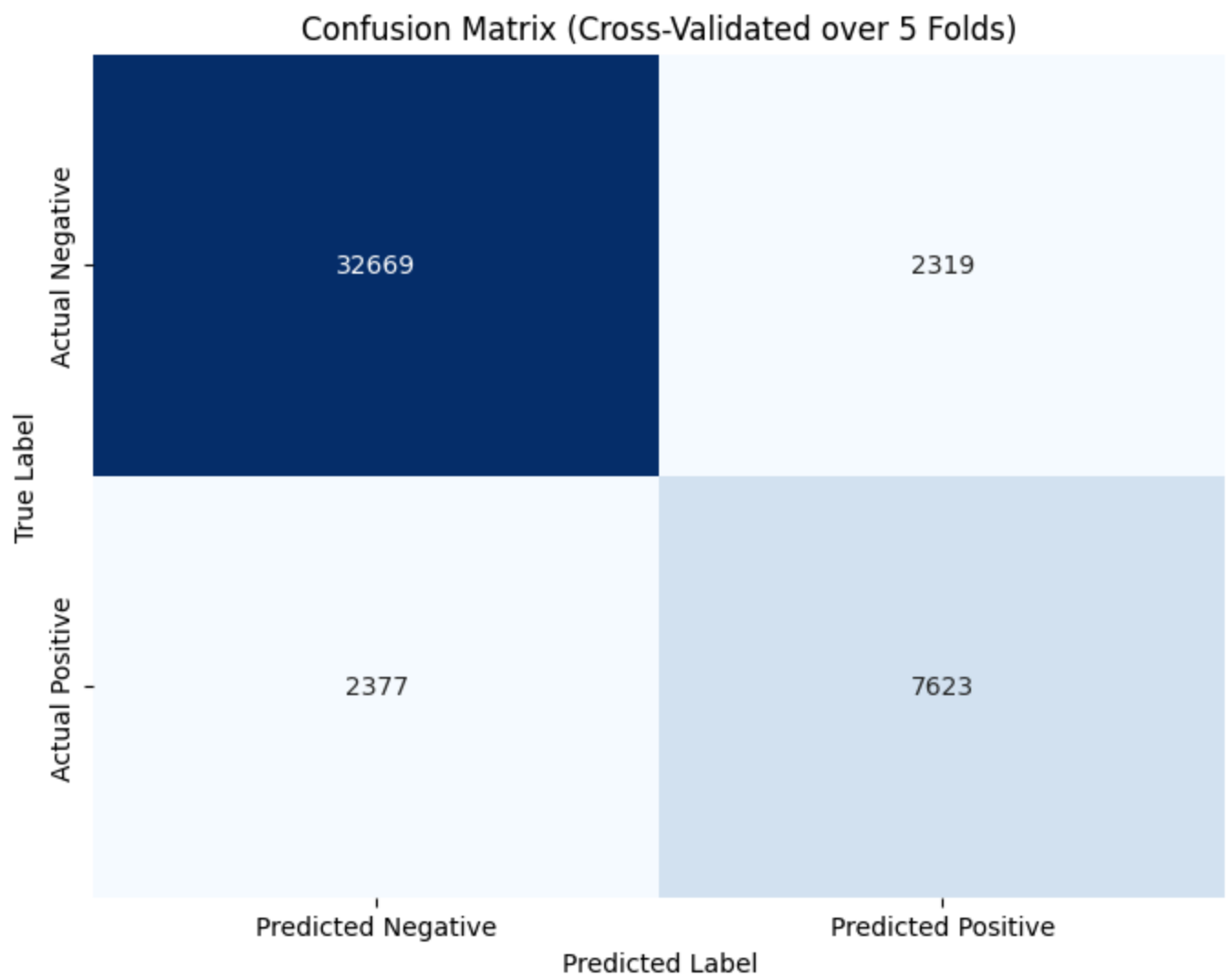
y_pred = cross_val_predict(log_reg, X, y, cv=skf)

conf = confusion_matrix(y, y_pred)
print("\nClassification Report:\n", classification_report(y, y_pred))

# Plot
plt.figure(figsize=(8, 6))
sns.heatmap(conf, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=["Predicted Negative", "Predicted Positive"],
            yticklabels=["Actual Negative", "Actual Positive"])
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title(f"Confusion Matrix (Cross-Validated over {skf.get_n_splits()} Folds)")
plt.show()
```

Classification Report:

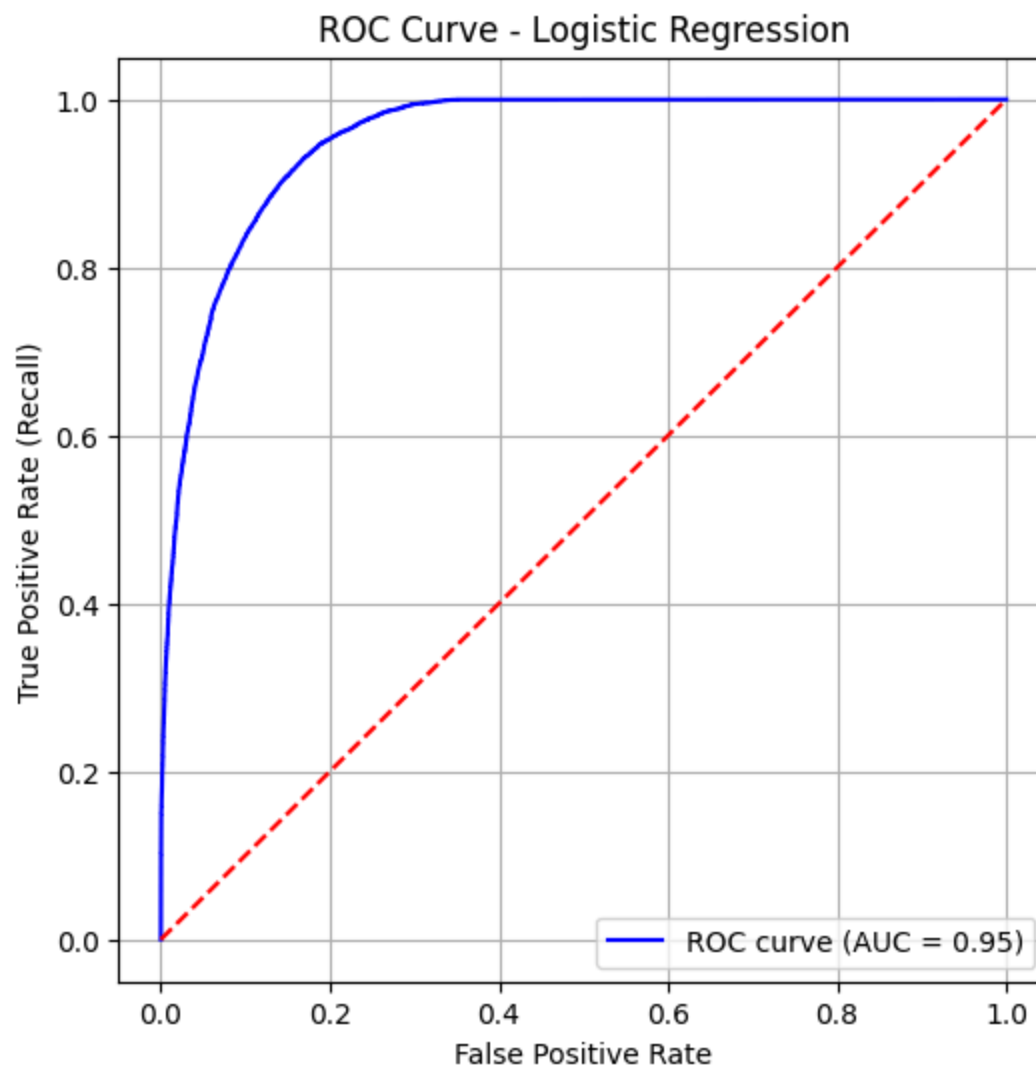
	precision	recall	f1-score	support
0	0.93	0.93	0.93	34988
1	0.77	0.76	0.76	10000
accuracy			0.90	44988
macro avg	0.85	0.85	0.85	44988
weighted avg	0.90	0.90	0.90	44988



```
In [51]: y_pred_proba = cross_val_predict(log_reg, X, y, cv=skf, method='predict_proba')[:, 1]

fpr, tpr, thresholds = roc_curve(y, y_pred_proba)
auc_score = roc_auc_score(y, y_pred_proba)

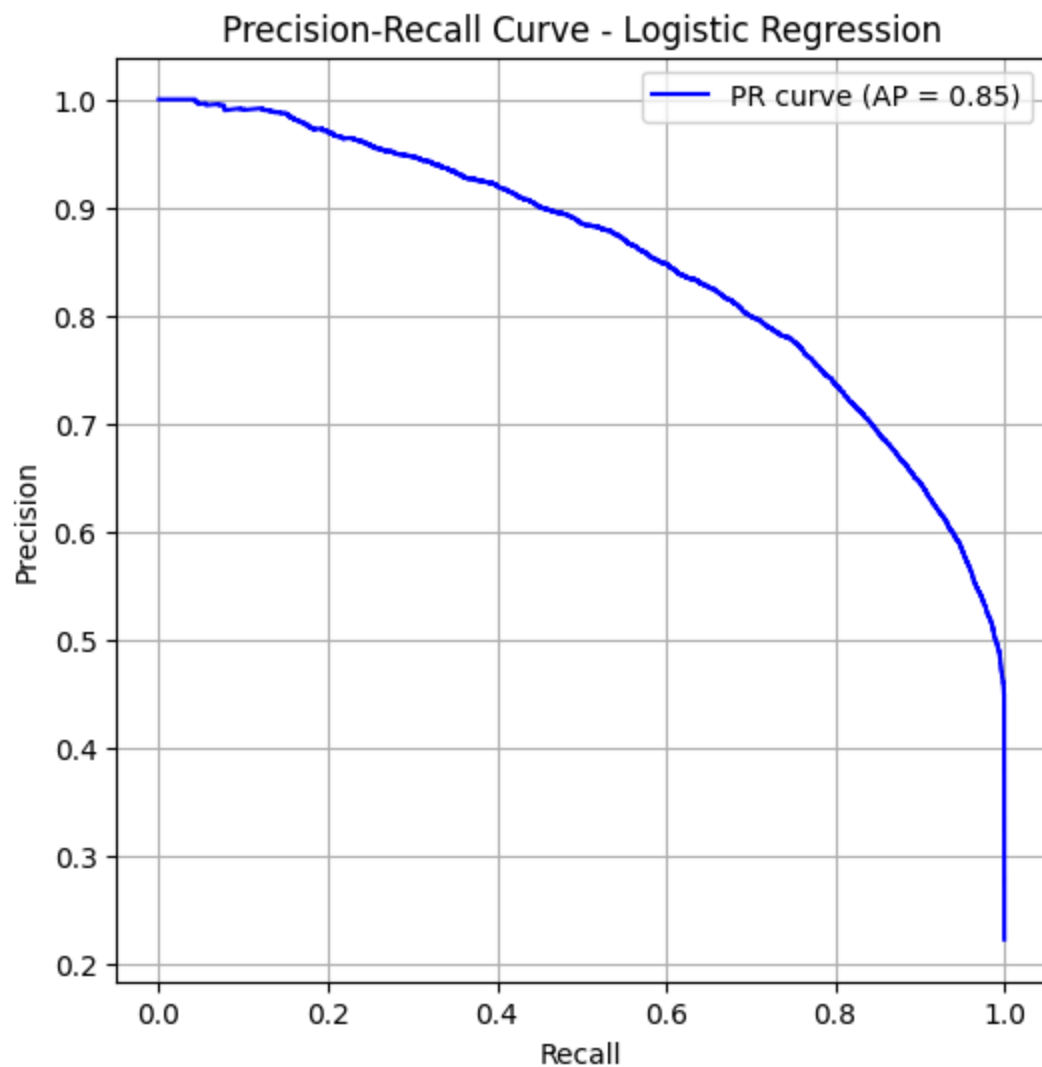
plt.figure(figsize=(6,6))
plt.plot(fpr, tpr, color="blue", label=f"ROC curve (AUC = {auc_score:.2f})")
plt.plot([0, 1], [0, 1], color="red", linestyle="--") # Random guess line
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate (Recall)")
plt.title("ROC Curve - Logistic Regression")
plt.legend()
plt.grid(True)
plt.show()
```



In [52]: *# Plot Precision-Recall Curve*

```
precision, recall, thresholds = precision_recall_curve(y, y_pred_proba)
ap_score = average_precision_score(y, y_pred_proba)

plt.figure(figsize=(6,6))
plt.plot(recall, precision, color="blue", label=f"PR curve (AP = {ap_score:.2f})")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve - Logistic Regression")
plt.legend()
plt.grid(True)
plt.show()
```



```
In [54]: # Achieve Precision of 90%: find threshold with highest recall at precision >= 0.9
best = max(
    ((t, p, r) for p, r, t in zip(precision, recall, np.append(thresholds, 1)) if p >= 0.9),
    key=lambda x: x[2], # maximise recall
    default=None
)

if best:
    print(f"Threshold={best[0]:.2f}, Precision={best[1]:.2f}, Recall={best[2]:.2f}")
else:
    print("No threshold achieves precision >= 0.9")
```

Threshold=0.77, Precision=0.90, Recall=0.45

```
In [55]: # Evaluate model at new custom threshold

custom_threshold = 0.77

y_pred_custom = (y_pred_proba >= custom_threshold).astype(int)

print("Accuracy:", accuracy_score(y, y_pred_custom))
print("Confusion Matrix:\n", confusion_matrix(y, y_pred_custom))
print("Classification Report:\n", classification_report(y, y_pred_custom))
```

Accuracy: 0.8675202276162532

Confusion Matrix:

```
[[34478  510]
```

```
[ 5450 4550]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.99	0.92	34988
1	0.90	0.46	0.60	10000
accuracy			0.87	44988
macro avg	0.88	0.72	0.76	44988
weighted avg	0.87	0.87	0.85	44988

XGBoost Classifier with RandomSearchCV

Data Pre-Processing

```
In [56]: # We will first check if our categorical variables have an ordinal relationship with the target variable
# This can be done using Spearman's rank correlation for ordinal variables (which was found to be significant)
# So we will treat all categorical variables as nominal and use one-hot encoding
# So we will use the same data as for the logistic regression model
```

```
data1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 44988 entries, 0 to 44999
```

```
Data columns (total 22 columns):
```

#	Column	Non-Null Count	Dtype
0	person_age	44988 non-null	float64
1	loan_amnt	44988 non-null	float64
2	loan_int_rate	44988 non-null	float64
3	cb_person_cred_hist_length	44988 non-null	float64
4	credit_score	44988 non-null	int64
5	loan_status	44988 non-null	int64
6	Log_Income	44988 non-null	float64
7	Person_Age_Emp_Ratio	44988 non-null	float64
8	person_gender_female	44988 non-null	bool
9	person_education_Associate	44988 non-null	bool
10	person_education_Bachelor	44988 non-null	bool
11	person_education_Doctorate	44988 non-null	bool
12	person_education_Master	44988 non-null	bool
13	person_home_ownership_MORTGAGE	44988 non-null	bool
14	person_home_ownership_OTHER	44988 non-null	bool
15	person_home_ownership_OWN	44988 non-null	bool
16	loan_intent_DEBTCONSOLIDATION	44988 non-null	bool
17	loan_intent_HOMEIMPROVEMENT	44988 non-null	bool
18	loan_intent_MEDICAL	44988 non-null	bool
19	loan_intent_PERSONAL	44988 non-null	bool
20	loan_intent_VENTURE	44988 non-null	bool
21	previous_loan_defaults_on_file_No	44988 non-null	bool

```
dtypes: bool(14), float64(6), int64(2)
```

```
memory usage: 4.7 MB
```

```
In [57]: X1, y1 = data1.drop(columns=["loan_status"]), data["loan_status"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X1, y1, test_size=0.2, random_state=42, stra
```

In [59]: *# Accuracy under scoring = 'roc_auc' was about 0.93 but gave a lower precision score. If we aimed*

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("xgb", XGBClassifier(eval_metric='logloss', random_state=42, objective='binary:logistic'))
])

param_distributions = {
    'xgb_n_estimators': randint(100, 800),           # number of boosting rounds
    'xgb_max_depth': randint(3, 10),                 # Max depth of a tree
    'xgb_learning_rate': loguniform(0.01, 0.3),      # scale contribution of each new tree
    'xgb_subsample': uniform(0.5, 0.5),              # sub sample of training data in every l
    'xgb_colsample_bytree': uniform(0.5, 0.5),       # fraction of features sampled to consti
    'xgb_gamma': uniform(0, 5),                      # penalty for splitting
    'xgb_reg_alpha': loguniform(0.001, 10),          # L1 penalty on leaf weights
    'xgb_reg_lambda': loguniform(0.01, 100),         # L2 penalty on leaf weights
    'xgb_min_child_weight': loguniform(0.1, 10),     # Minimum sum of instance weight needed
    'xgb_grow_policy': ['depthwise', 'lossguide']     # Depth-first vs Loss reduction guided
}

random_search = RandomizedSearchCV(
    estimator=pipe,
    param_distributions=param_distributions,
    scoring = make_scorer(precision_score),          # Use precision as the scoring metric
    cv=5,                                           # 5-fold cross-validation
    verbose=2,                                     # Level of message verbosity
    random_state=42,                               # Random seed for reproducibility
    n_jobs=-1,                                     # Use all available cores
    n_iter=100
)

random_search.fit(X_train, y_train)

best_model = random_search.best_estimator_
test_score = best_model.score(X_test, y_test)

hide_output = True
```

Fitting 5 folds for each of 100 candidates, totalling 500 fits

```
In [60]: y_train_pred = best_model.predict(X_train)
y_test_pred = best_model.predict(X_test)

print("Train Accuracy:", accuracy_score(y_train, y_train_pred))
print("Test Accuracy:", accuracy_score(y_test, y_test_pred))

cm_train = confusion_matrix(y_train, y_train_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_train, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=["Predicted Negative", "Predicted Positive"],
            yticklabels=["Actual Negative", "Actual Positive"])

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix Heatmap Train")
plt.show()

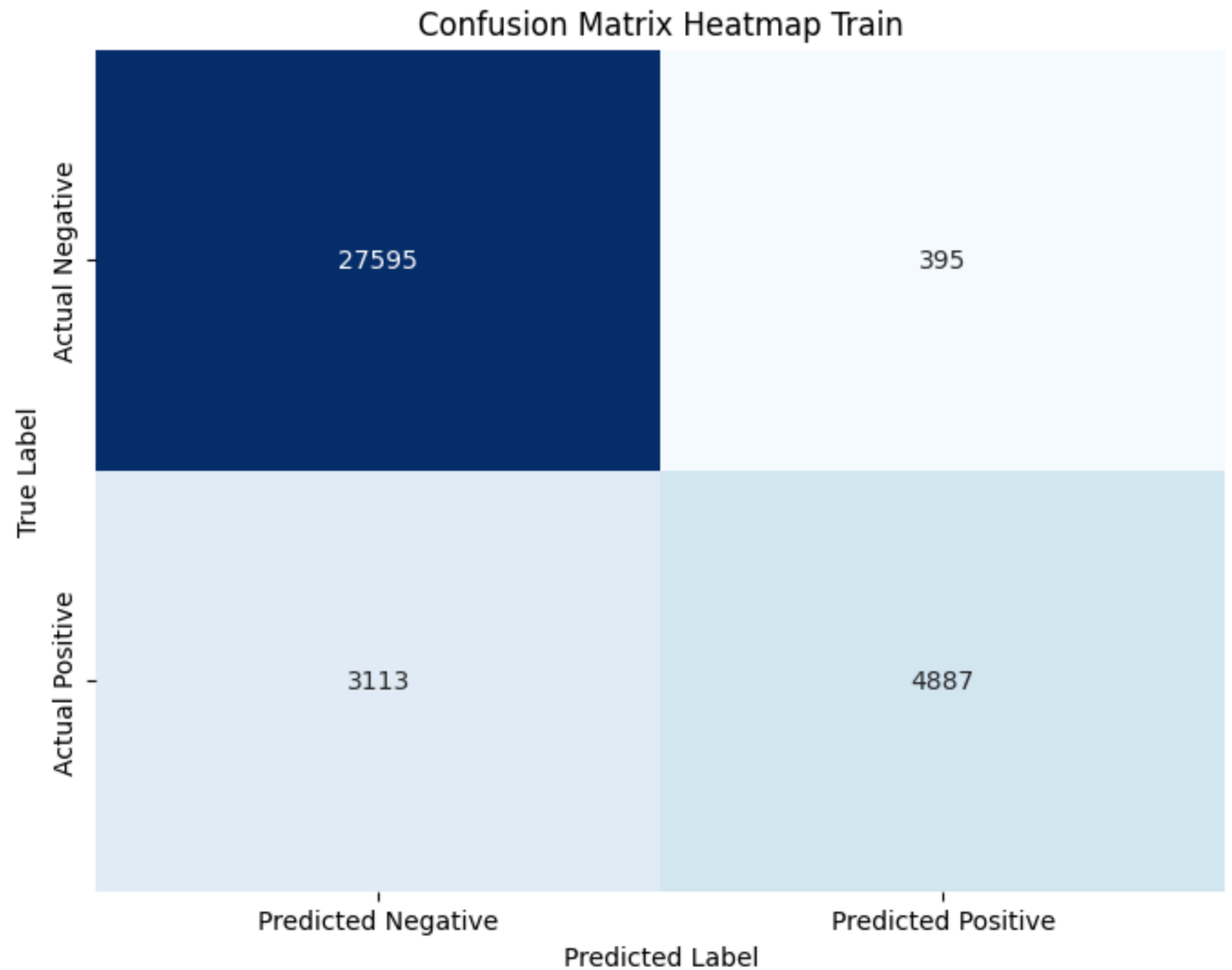
cm_test = confusion_matrix(y_test, y_test_pred)
```

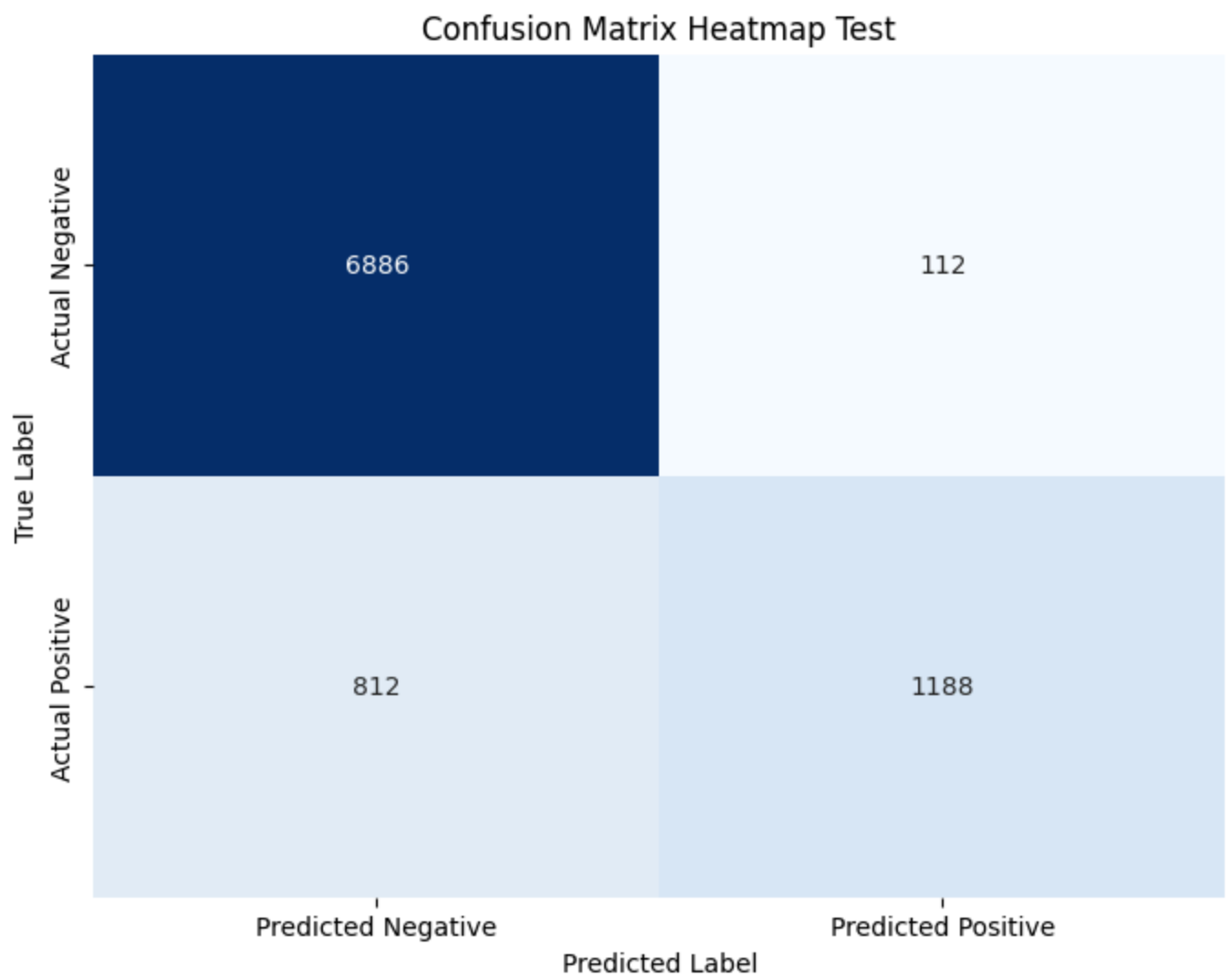
```
plt.figure(figsize=(8, 6))
sns.heatmap(cm_test, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=["Predicted Negative", "Predicted Positive"],
            yticklabels=["Actual Negative", "Actual Positive"])

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix Heatmap Test")
plt.show()
```

Train Accuracy: 0.9025284801333704

Test Accuracy: 0.8973105134474327





```
In [30]: print("Train Classification Report:\n", classification_report(y_train, y_train_pred))
print("Test Classification Report:\n", classification_report(y_test, y_test_pred))
```

Train Classification Report:

	precision	recall	f1-score	support
0	0.90	0.99	0.94	27990
1	0.93	0.61	0.74	8000
accuracy			0.90	35990
macro avg	0.91	0.80	0.84	35990
weighted avg	0.90	0.90	0.89	35990

Test Classification Report:

	precision	recall	f1-score	support
0	0.89	0.98	0.94	6998
1	0.91	0.59	0.72	2000
accuracy			0.90	8998
macro avg	0.90	0.79	0.83	8998
weighted avg	0.90	0.90	0.89	8998

RandomForest Classifier

```

In [32]: rf = RandomForestClassifier(n_estimators=50, random_state=42)

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'bootstrap': [True, False]
}

rf_cv = RandomizedSearchCV(
    rf,
    param_grid,
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42),
    n_iter=30,
    scoring='roc_auc',
    n_jobs=-1,
    verbose=1
)
rf_cv.fit(X_train, y_train)

y_train_pred = rf_cv.predict(X_train)
y_test_pred = rf_cv.predict(X_test)

print("Train Classification Report:\n", classification_report(y_train, y_train_pred))
print("Test Classification Report:\n", classification_report(y_test, y_test_pred))

print("Best Parameters:", rf_cv.best_params_)
best_model = rf_cv.best_estimator_
accuracy = best_model.score(X_test, y_test)
print("Tuned Random Forest Accuracy:", accuracy)

```

Fitting 3 folds for each of 30 candidates, totalling 90 fits

Train Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	1.00	27990
1	1.00	0.98	0.99	8000
accuracy			0.99	35990
macro avg	1.00	0.99	0.99	35990
weighted avg	0.99	0.99	0.99	35990

Test Classification Report:

	precision	recall	f1-score	support
0	0.94	0.97	0.95	6998
1	0.88	0.77	0.82	2000
accuracy			0.93	8998
macro avg	0.91	0.87	0.89	8998
weighted avg	0.92	0.93	0.92	8998

Best Parameters: {'n_estimators': 300, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_depth': 20, 'bootstrap': False}

Tuned Random Forest Accuracy: 0.925872416092465

```

In [33]: cm_train = confusion_matrix(y_train, y_train_pred)
plt.figure(figsize=(8, 6))

```

```

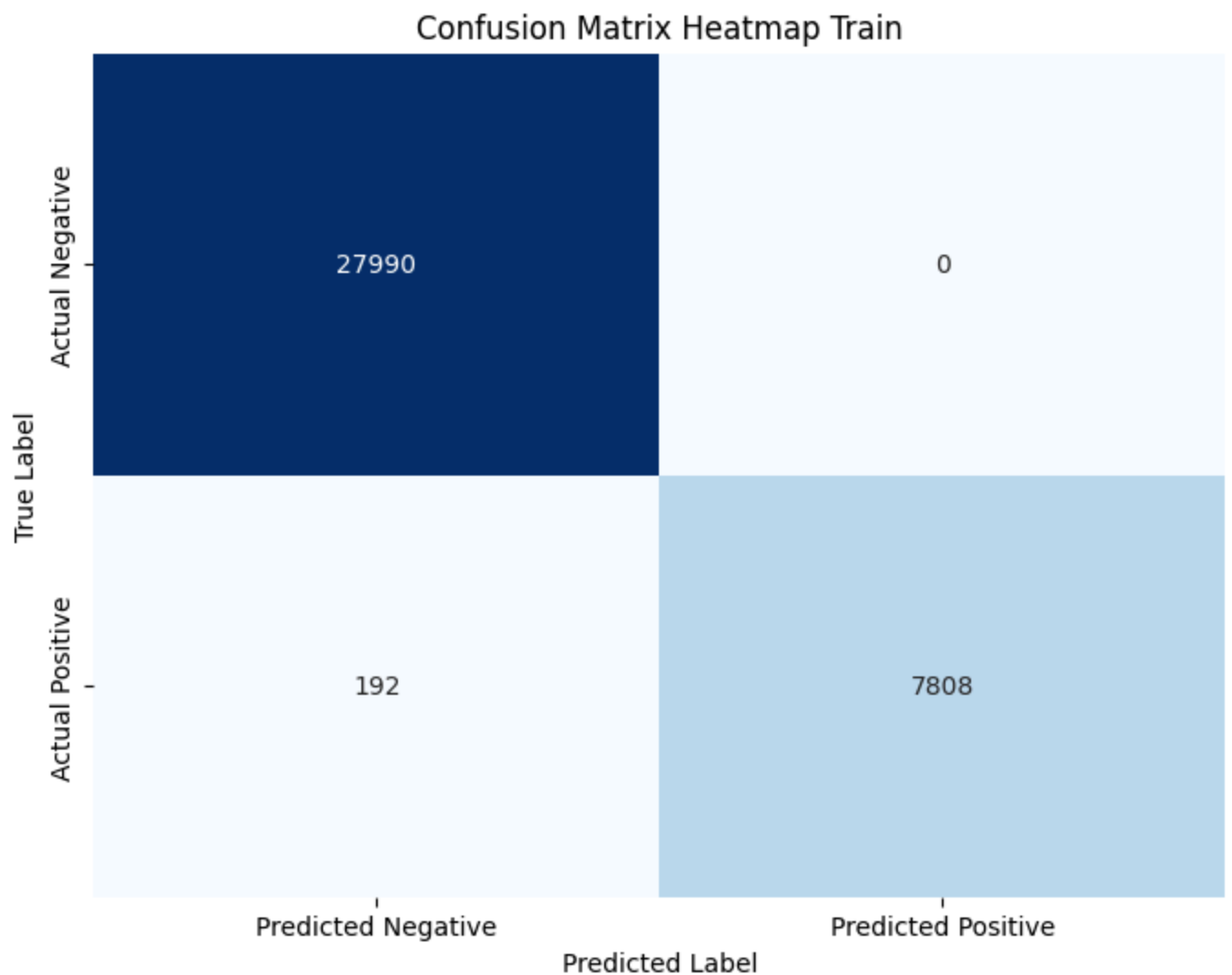
sns.heatmap(cm_train, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=["Predicted Negative", "Predicted Positive"],
            yticklabels=["Actual Negative", "Actual Positive"])

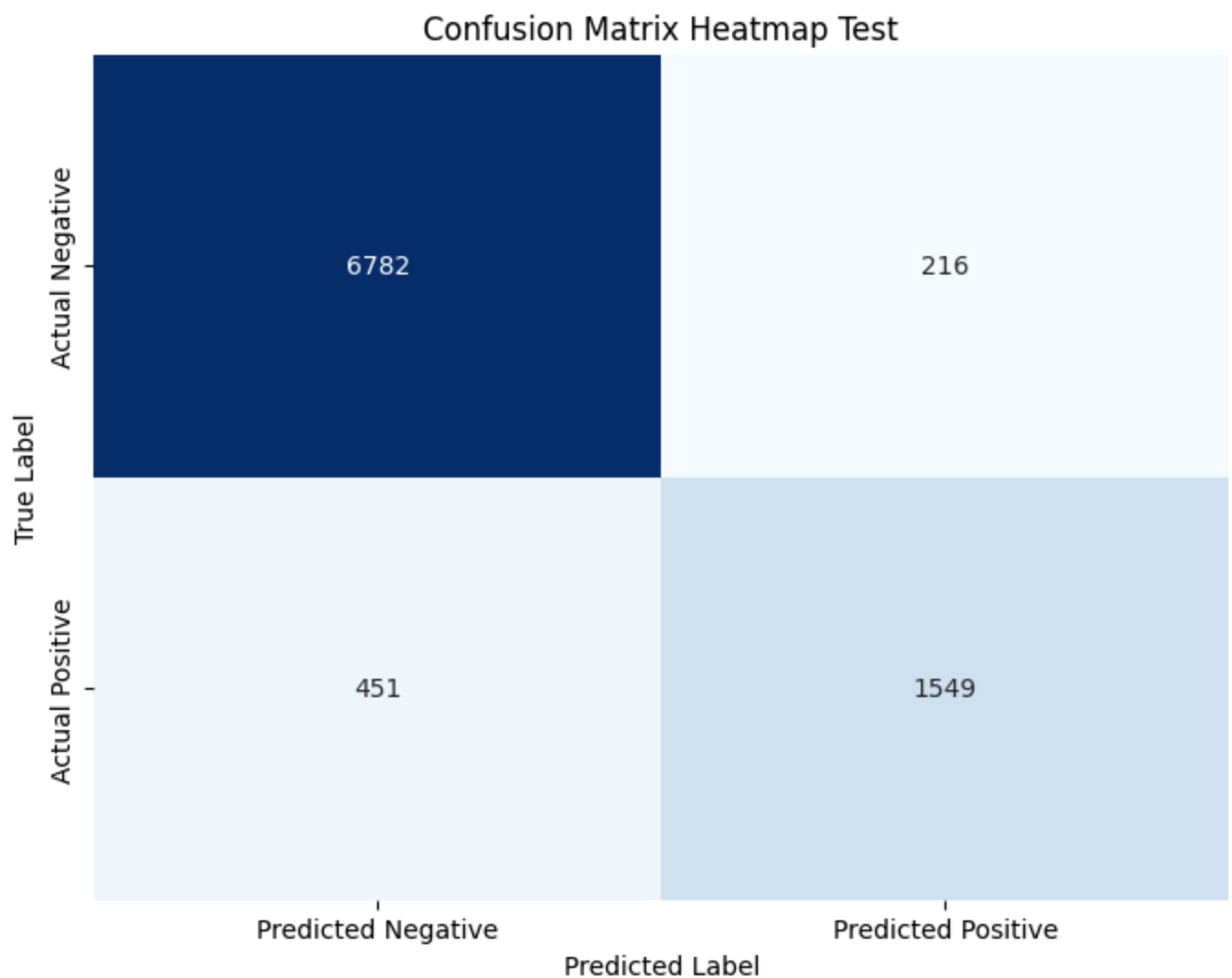
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix Heatmap Train")
plt.show()

cm_test = confusion_matrix(y_test, y_test_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_test, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=["Predicted Negative", "Predicted Positive"],
            yticklabels=["Actual Negative", "Actual Positive"])

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix Heatmap Test")
plt.show()

```





Artificial Neural Network

```
In [ ]: from imblearn.over_sampling import SMOTE
        from imblearn.pipeline import Pipeline as ImbPipeline

        mlp = MLPClassifier(
            max_iter = 500,
            early_stopping=True,
            random_state=42
        )

        pipeline = ImbPipeline(steps=[
            ('smote', SMOTE(random_state=42)), # SMOTE to handle class imbalance
            ('scaler', StandardScaler()),
            ('mlp', mlp)
        ])

        param_grid = {
            'mlp__hidden_layer_sizes': [(32,), (64,), (64, 32)], # number of neurons in hidden layers
            'mlp__activation': ['relu', 'tanh'], # activation function for hidden layers
            'mlp__alpha': [0.0001, 0.001, 0.01], # L2 regularization parameter
            'mlp__learning_rate_init': [0.0005, 0.001, 0.005], # Learning rate for weight updates
        }

        grid_search = RandomizedSearchCV(
```

```

pipeline,
param_grid,
cv=5,
scoring='roc_auc',
n_jobs=-1,
verbose=2
)
grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_

y_train_pred = best_model.predict(X_train)
y_test_pred = best_model.predict(X_test)

print("Best Parameters:", grid_search.best_params_)
print("ROC AUC Score:", roc_auc_score(y_test, best_model.predict_proba(X_test)[: , 1]))
print("Train Classification Report:\n", classification_report(y_train, y_train_pred))

```

Fitting 5 folds for each of 10 candidates, totalling 50 fits

Best Parameters: {'mlp__learning_rate_init': 0.001, 'mlp__hidden_layer_sizes': (64, 32), 'mlp__alpha': 0.01, 'mlp__activation': 'tanh'}

ROC AUC Score: 0.9653078736781938

Train Classification Report:

	precision	recall	f1-score	support
0	0.96	0.93	0.94	27990
1	0.78	0.85	0.81	8000
accuracy			0.91	35990
macro avg	0.87	0.89	0.88	35990
weighted avg	0.92	0.91	0.91	35990

In [44]: `print("Test Classification Report:\n", classification_report(y_test, y_test_pred))`

Test Classification Report:

	precision	recall	f1-score	support
0	0.95	0.92	0.94	6998
1	0.76	0.85	0.80	2000
accuracy			0.91	8998
macro avg	0.86	0.88	0.87	8998
weighted avg	0.91	0.91	0.91	8998

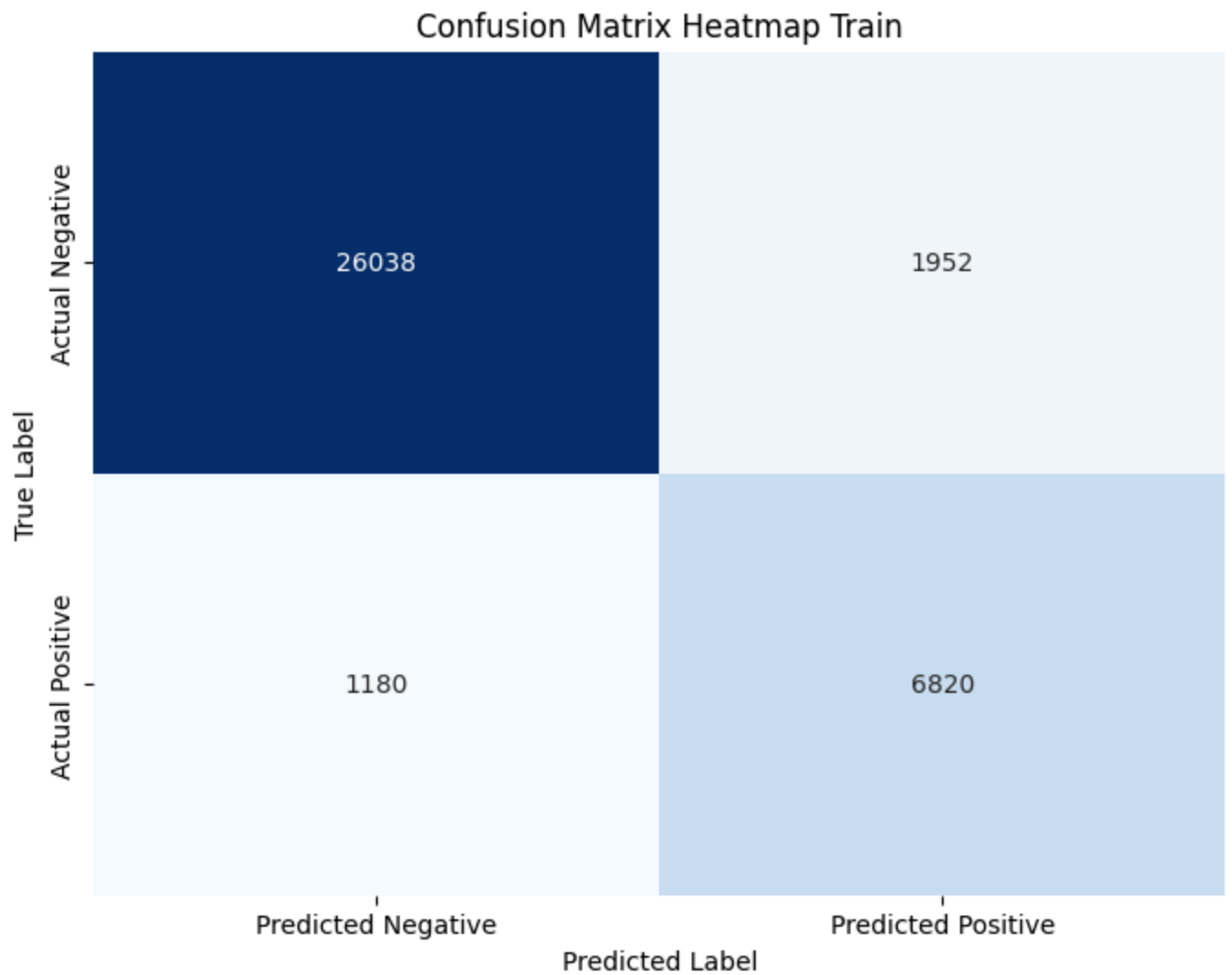
In [43]: `cm_train = confusion_matrix(y_train, y_train_pred)`
`plt.figure(figsize=(8, 6))`
`sns.heatmap(cm_train, annot=True, fmt="d", cmap="Blues", cbar=False,`
`xticklabels=["Predicted Negative", "Predicted Positive"],`
`yticklabels=["Actual Negative", "Actual Positive"])`

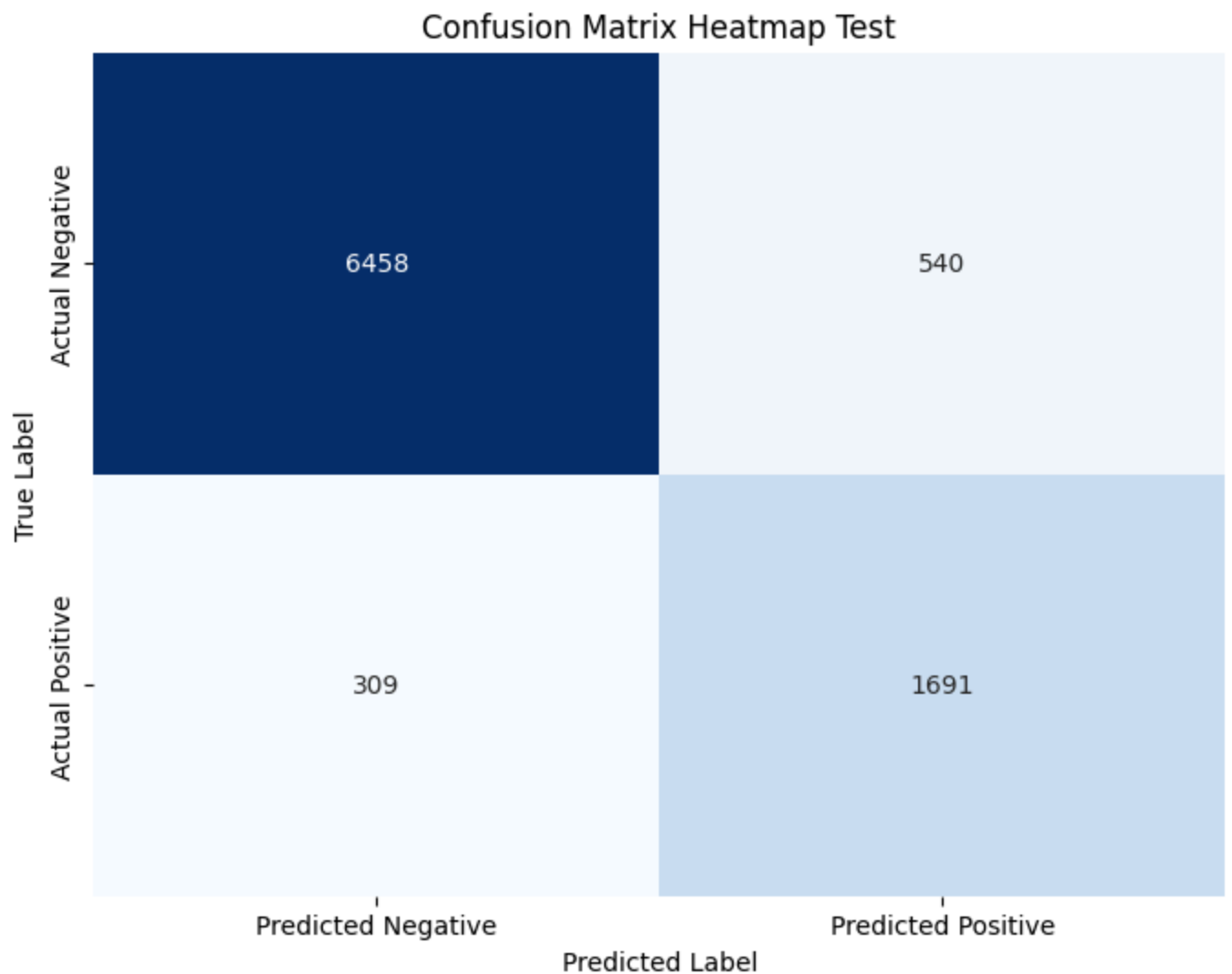
`plt.xlabel("Predicted Label")`
`plt.ylabel("True Label")`
`plt.title("Confusion Matrix Heatmap Train")`
`plt.show()`

`cm_test = confusion_matrix(y_test, y_test_pred)`
`plt.figure(figsize=(8, 6))`
`sns.heatmap(cm_test, annot=True, fmt="d", cmap="Blues", cbar=False,`

```
xticklabels=["Predicted Negative", "Predicted Positive"],  
yticklabels=["Actual Negative", "Actual Positive"])
```

```
plt.xlabel("Predicted Label")  
plt.ylabel("True Label")  
plt.title("Confusion Matrix Heatmap Test")  
plt.show()
```





Model Comparison

```
In [ ]: # === Model Comparison Dashboard ===
# Re-fit logistic regression on the same train/test split for a fair comparison
log_reg.fit(X_train, y_train)

# Collect all models and their names
models = {
    'Logistic Regression': log_reg,
    'XGBoost': random_search.best_estimator_,
    'Random Forest': rf_cv.best_estimator_,
    'Neural Network (MLP)': grid_search.best_estimator_
}

# Compute metrics for each model on the test set
results = []
roc_data = {}
pr_data = {}

for name, model in models.items():
    y_pred = model.predict(X_test)

    if hasattr(model, 'predict_proba'):
        y_proba = model.predict_proba(X_test)[:, 1]
    else:
```

```

y_proba = model.decision_function(X_test)

report = classification_report(y_test, y_pred, output_dict=True)

results.append({
    'Model': name,
    'Accuracy': accuracy_score(y_test, y_pred),
    'Precision (Class 1)': report['1']['precision'],
    'Recall (Class 1)': report['1']['recall'],
    'F1-Score (Class 1)': report['1']['f1-score'],
    'ROC AUC': roc_auc_score(y_test, y_proba),
    'Avg Precision (AP)': average_precision_score(y_test, y_proba)
})

fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_data[name] = (fpr, tpr)

prec, rec, _ = precision_recall_curve(y_test, y_proba)
pr_data[name] = (rec, prec)

results_df = pd.DataFrame(results).set_index('Model')
colors = ['#2196F3', '#FF9800', '#4CAF50', '#9C27B0']

# ---- Plot 1: Grouped Bar Chart – Key Metrics ----
fig, ax1 = plt.subplots(figsize=(14, 6))
metrics_to_plot = ['Accuracy', 'Precision (Class 1)', 'Recall (Class 1)', 'F1-Score (Class 1)']
x = np.arange(len(metrics_to_plot))
width = 0.18
for i, (model_name, row) in enumerate(results_df.iterrows()):
    vals = [row[m] for m in metrics_to_plot]
    bars = ax1.bar(x + i * width, vals, width, label=model_name, color=colors[i], edgecolor='white')
    for bar, val in zip(bars, vals):
        ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.008, f'{val:.3f}',
                  ha='center', va='bottom', fontsize=9, fontweight='bold')
ax1.set_xticks(x + width * 1.5)
ax1.set_xticklabels(['Accuracy', 'Precision', 'Recall', 'F1-Score'], fontsize=12)
ax1.set_ylim(0, 1.12)
ax1.set_title('Classification Metrics (Class 1 – Approved)', fontsize=16, fontweight='bold')
ax1.legend(fontsize=10, loc='lower right')
ax1.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

# ---- Plot 2: ROC Curves Overlay ----
fig, ax2 = plt.subplots(figsize=(10, 8))
for i, (name, (fpr_vals, tpr_vals)) in enumerate(roc_data.items()):
    auc_val = results_df.loc[name, 'ROC AUC']
    ax2.plot(fpr_vals, tpr_vals, color=colors[i], linewidth=2.5, label=f'{name} (AUC={auc_val:.3f})')
ax2.plot([0, 1], [0, 1], 'k--', alpha=0.4, label='Random Guess')
ax2.set_xlabel('False Positive Rate', fontsize=13)
ax2.set_ylabel('True Positive Rate', fontsize=13)
ax2.set_title('ROC Curves', fontsize=16, fontweight='bold')
ax2.legend(fontsize=11)
ax2.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# ---- Plot 3: Precision-Recall Curves Overlay ----
fig, ax3 = plt.subplots(figsize=(10, 8))

```

```

for i, (name, (rec_vals, prec_vals)) in enumerate(pr_data.items()):
    ap_val = results_df.loc[name, 'Avg Precision (AP)']
    ax3.plot(rec_vals, prec_vals, color=colors[i], linewidth=2.5, label=f'{name} (AP={ap_val:.3f})')
ax3.set_xlabel('Recall', fontsize=13)
ax3.set_ylabel('Precision', fontsize=13)
ax3.set_title('Precision-Recall Curves', fontsize=16, fontweight='bold')
ax3.legend(fontsize=11)
ax3.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# ---- Plot 4: ROC AUC & Average Precision (Horizontal Bar) ----
fig, ax4 = plt.subplots(figsize=(12, 5))
model_names = results_df.index.tolist()
auc_vals = results_df['ROC AUC'].values
ap_vals = results_df['Avg Precision (AP)'].values
y_pos = np.arange(len(model_names))
bar_h = 0.35
ax4.barh(y_pos - bar_h/2, auc_vals, bar_h, label='ROC AUC', color='#1565C0', edgecolor='white')
ax4.barh(y_pos + bar_h/2, ap_vals, bar_h, label='Avg Precision', color='#E65100', edgecolor='white')
for i in range(len(model_names)):
    ax4.text(auc_vals[i] + 0.005, y_pos[i] - bar_h/2, f'{auc_vals[i]:.3f}', va='center', fontsize=10)
    ax4.text(ap_vals[i] + 0.005, y_pos[i] + bar_h/2, f'{ap_vals[i]:.3f}', va='center', fontsize=10)
ax4.set_yticks(y_pos)
ax4.set_yticklabels(model_names, fontsize=12)
ax4.set_title('ROC AUC & Average Precision', fontsize=16, fontweight='bold')
ax4.legend(fontsize=11)
ax4.set_xlim(0, 1.15)
ax4.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

# ---- Plot 5: Confusion Matrices (2x2 grid) ----
fig, cm_axes = plt.subplots(2, 2, figsize=(14, 11))
fig.suptitle('Confusion Matrices (Test Set)', fontsize=16, fontweight='bold')
for i, (name, model) in enumerate(models.items()):
    ax = cm_axes[i // 2, i % 2]
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=ax,
                annot_kws={'size': 14},
                xticklabels=['Predicted Neg', 'Predicted Pos'],
                yticklabels=['Actual Neg', 'Actual Pos'])
    ax.set_title(name, fontsize=13, fontweight='bold')
    ax.tick_params(labelsize=11)
plt.tight_layout()
plt.show()

# ---- Plot 6: Summary Table ----
fig, ax6 = plt.subplots(figsize=(16, 4))
ax6.axis('off')
table_data = results_df.round(4).reset_index()
table = ax6.table(
    cellText=table_data.values,
    colLabels=table_data.columns,
    cellLoc='center',
    loc='center'
)
table.auto_set_font_size(False)

```

```

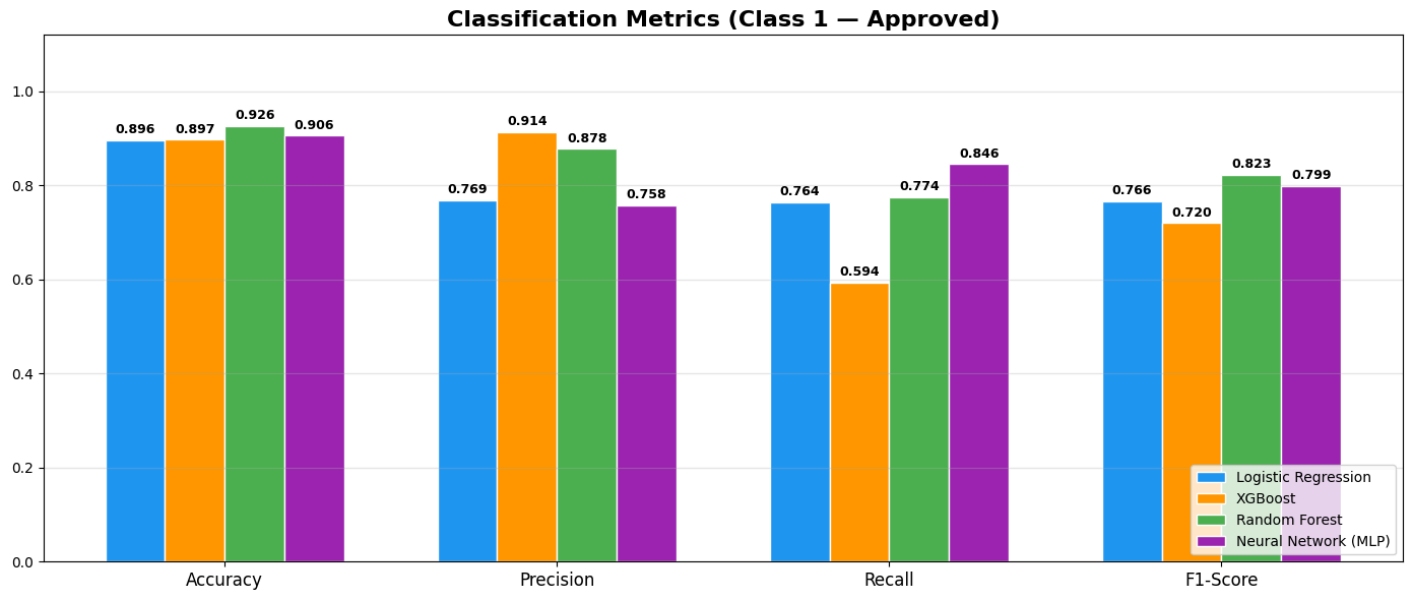
table.set_fontsize(11)
table.scale(1.2, 2.0)
a
for j in range(len(table_data.columns)):
    table[0, j].set_facecolor('#1565C0')
    table[0, j].set_text_props(color='white', fontweight='bold')

for i in range(1, len(table_data) + 1):
    for j in range(len(table_data.columns)):
        if i % 2 == 0:
            table[i, j].set_facecolor('#E3F2FD')

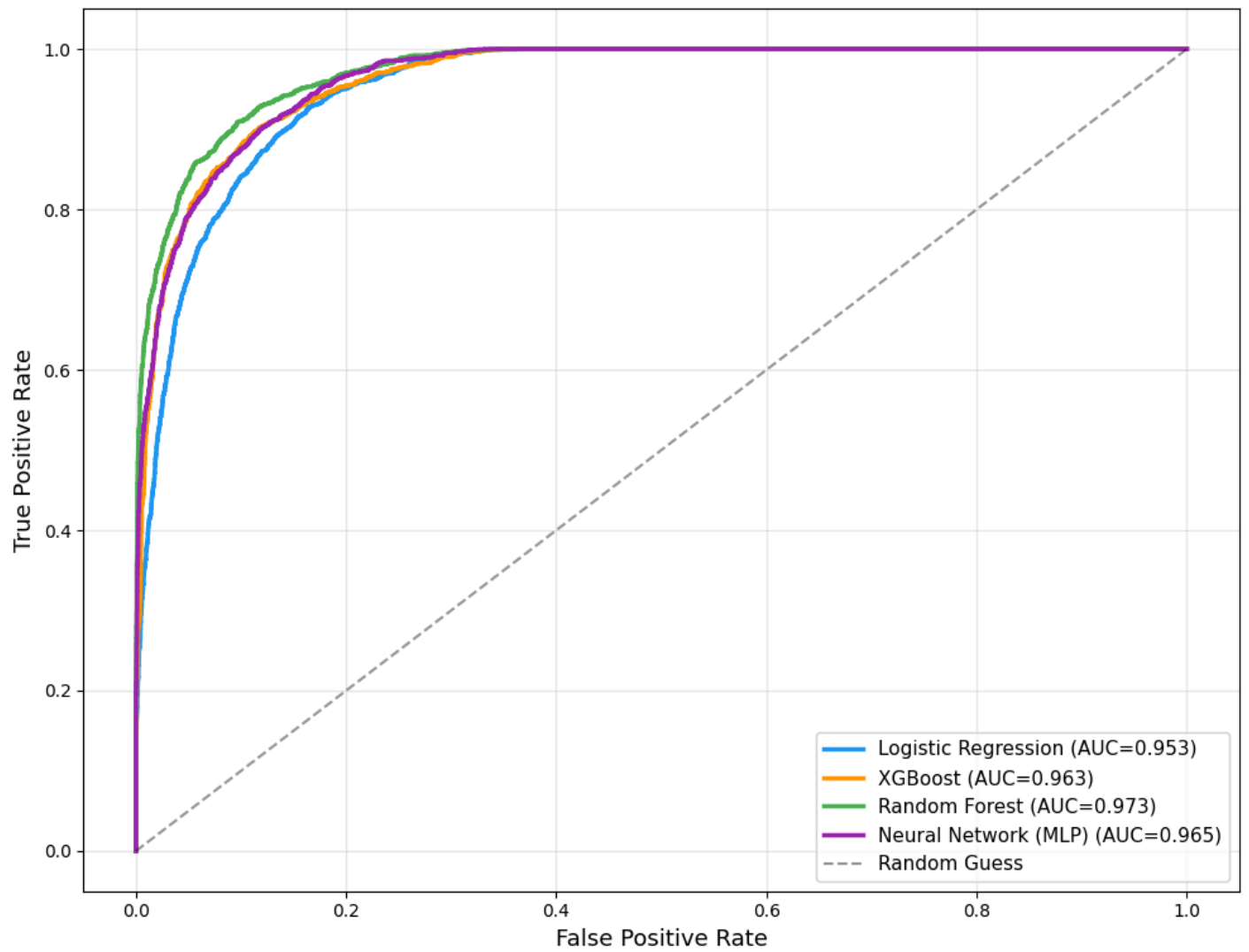
ax6.set_title('Model Comparison Summary', fontsize=16, fontweight='bold', pad=30)
plt.tight_layout()
plt.show()

# Print summary
print("\n" + "="*90)
print("MODEL COMPARISON SUMMARY")
print("="*90)
print(results_df.round(4).to_string())
print("="*90)

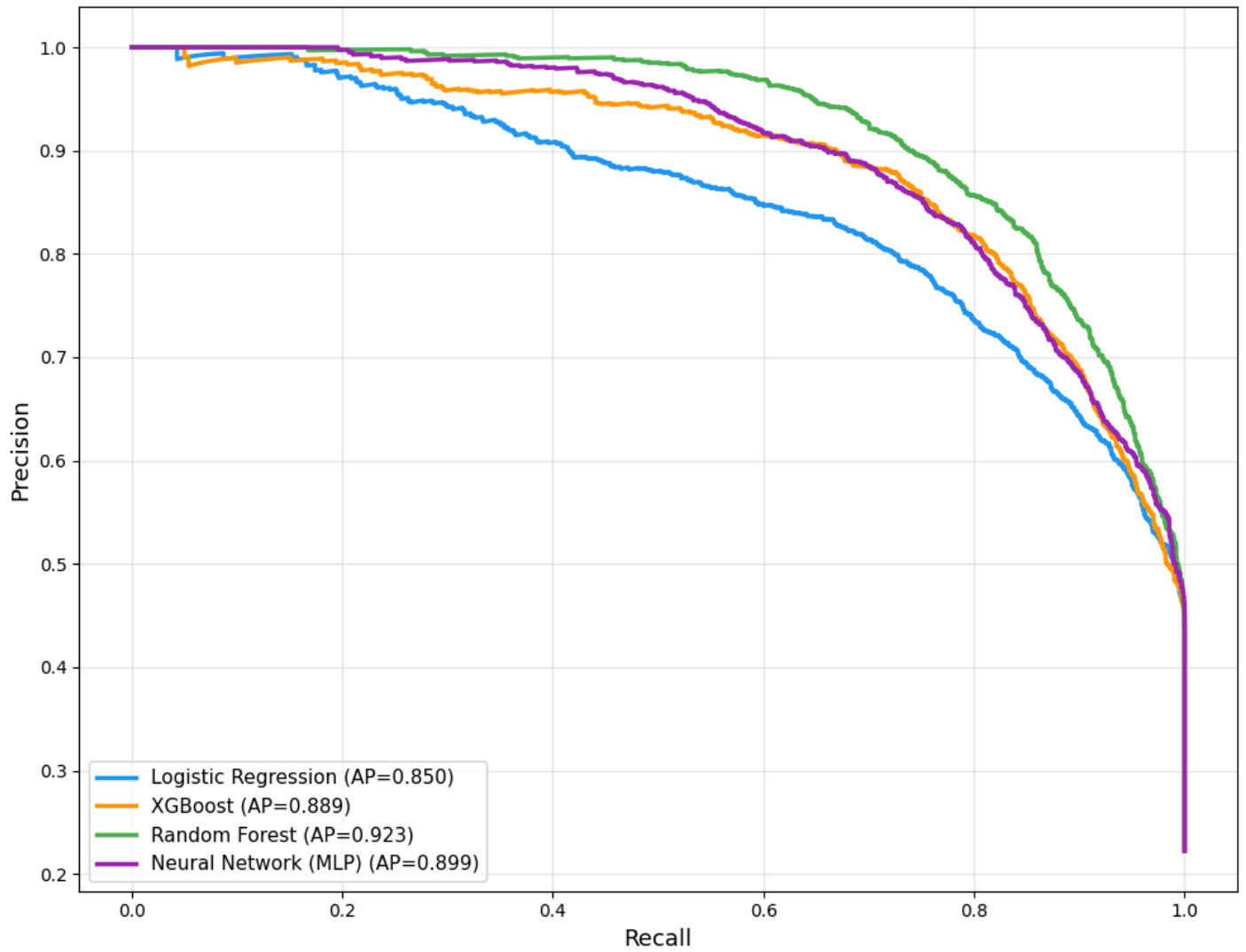
```



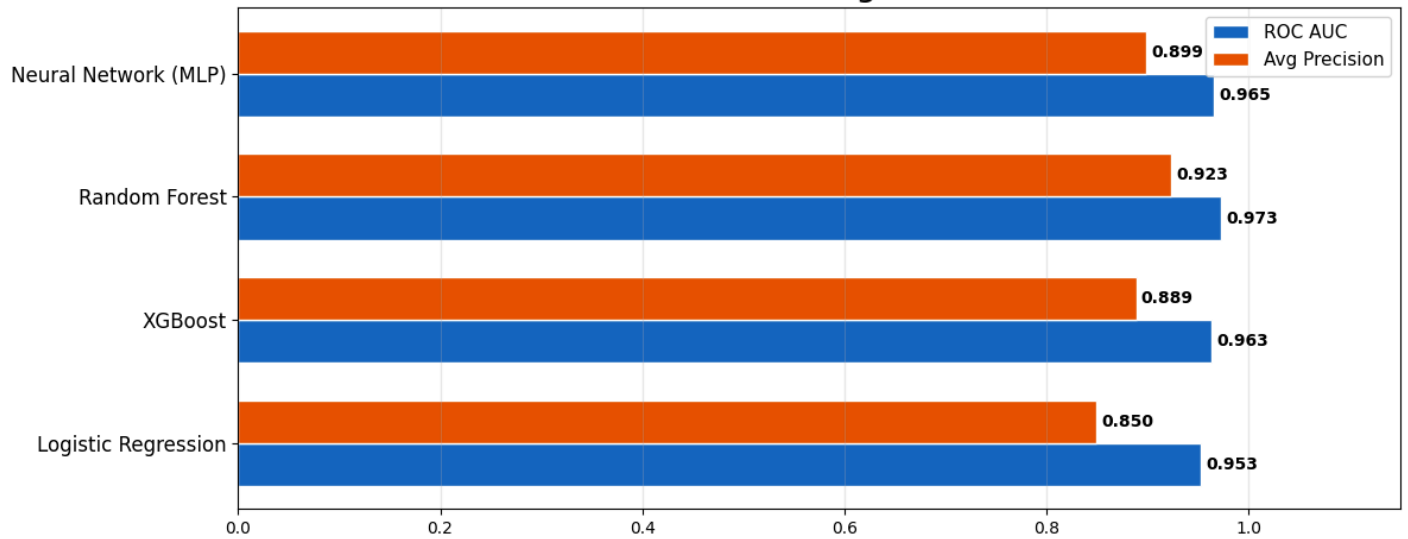
ROC Curves



Precision-Recall Curves



ROC AUC & Average Precision



The figure displays four confusion matrices for different machine learning models, arranged in a 2x2 grid. Each matrix compares the model's predictions against the actual outcomes (Negative or Positive). The color scale indicates the count of instances, with darker blue representing higher counts.

Logistic Regression

	Predicted Neg	Predicted Pos
Actual Neg	6538	460
Actual Pos	472	1528

XGBoost

	Predicted Neg	Predicted Pos
Actual Neg	6886	112
Actual Pos	812	1188

Random Forest

	Predicted Neg	Predicted Pos
Actual Neg	6782	216
Actual Pos	451	1549

Neural Network (MLP)

	Predicted Neg	Predicted Pos
Actual Neg	6458	540
Actual Pos	309	1691

Model	Accuracy	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)	ROC AUC	Avg Precision (AP)
Logistic Regression	0.8964	0.7686	0.764	0.7663	0.953	0.8496
XGBoost	0.8973	0.9138	0.594	0.72	0.9634	0.8886
Random Forest	0.9259	0.8776	0.7745	0.8228	0.9727	0.9228
Neural Network (MLP)	0.9056	0.758	0.8455	0.7993	0.9653	0.8986

Model Comparison Summary						
		Accuracy	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)	ROC AU
C	Avg Precision (AP)					
Model						
Logistic Regression		0.8964	0.7686	0.7640	0.7663	0.953
0	0.8496					
XGBoost		0.8973	0.9138	0.5940	0.7200	0.963
4	0.8886					
Random Forest		0.9259	0.8776	0.7745	0.8228	0.972
7	0.9228					
Neural Network (MLP)		0.9056	0.7580	0.8455	0.7993	0.965
3	0.8986					